AI Agents & Agentic AI Masterclass

Building Autonomous AI Systems in 2026

1. AI Agents
2. Agentic Architectures
3. Modern Prompting
4. Design Patterns
5. Tool-Using Agents
6. Production AI Stack

Instructor: **Nisarg Kadam**

The Evolution of AI Systems

Rule-Based Systems

Traditional AI with fixed logic and deterministic outputs.

Machine Learning

Predictive models trained on data patterns.

Generative AI

LLMs that generate rich, contextual text.

Agentic AI

Autonomous decision-making systems combining reasoning, planning, memory, tool usage, and execution loops.

AI agents transform LLMs from **chatbots → autonomous systems**.

What Is an AI Agent?

Definition

An **AI Agent** is a system that observes its environment, reasons about goals, plans actions, uses tools, and executes tasks autonomously — going far beyond a simple text response.

Chatbots → text response
Agents → **actions in the real world**

Real-World Examples

→ AI Coding Assistants

Write, test, and debug code autonomously.

→ Autonomous Research Agents

Search, synthesize, and report findings.

→ AI Workflow Automation

Orchestrate multi-step business processes.

What Is Agentic AI?

Systems where **multiple AI agents collaborate** to achieve complex goals — representing the transition from prompt-response systems to **goal-driven autonomous systems**.

Goal Decomposition

Break complex objectives into manageable sub-tasks.

Multi-Step Reasoning

Chain thoughts across extended planning horizons.

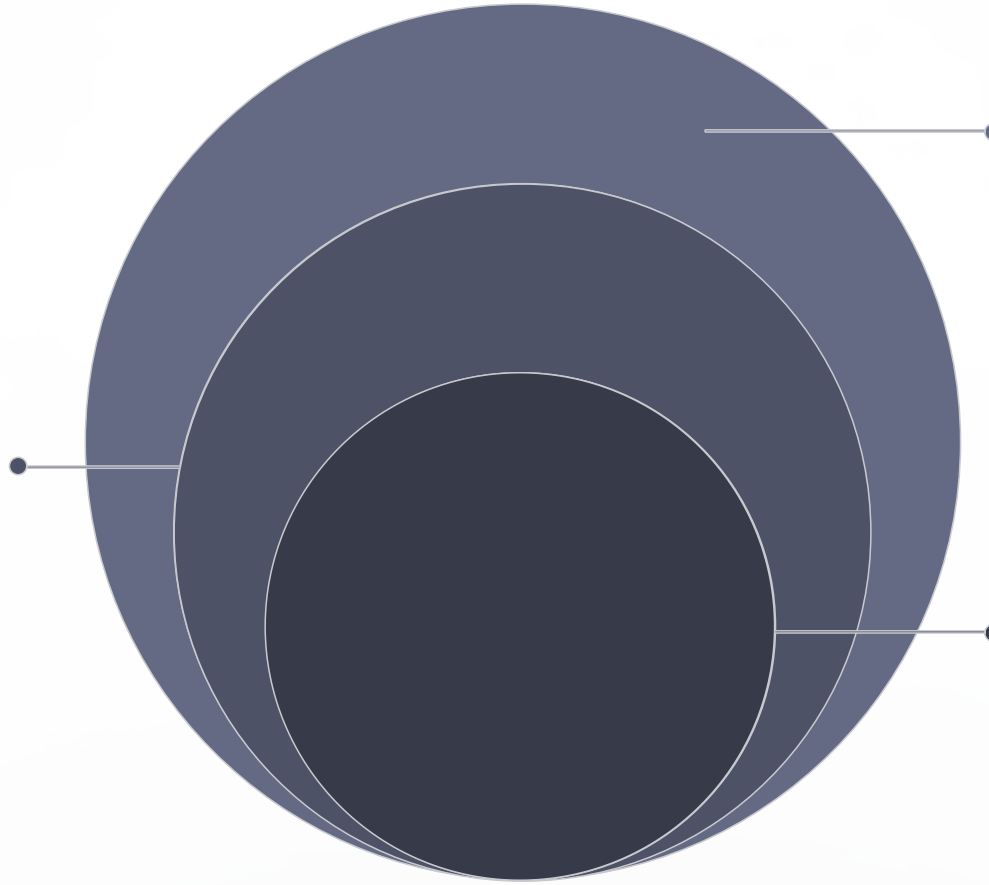
Memory Persistence

Retain context across sessions and tasks.

Multi-Agent Coordination

Agents collaborate, delegate, and orchestrate tools.

AI Agent Architecture



Interaction & Systems

User input and external systems

Execution Layer

Planner and tool executor

Core Reasoning

LLM for decision making

Every agent is built on these core components: **LLM** for reasoning, **Memory** for context, **Tools** for action, a **Planner** for strategy, an **Executor** for task completion, and a **Feedback Loop** for continuous improvement.

Modern AI Agent Stack

Frontend

- React
- Next.js

Backend

- Python
- FastAPI

Agent Frameworks

- LangGraph
- CrewAI
- AutoGen

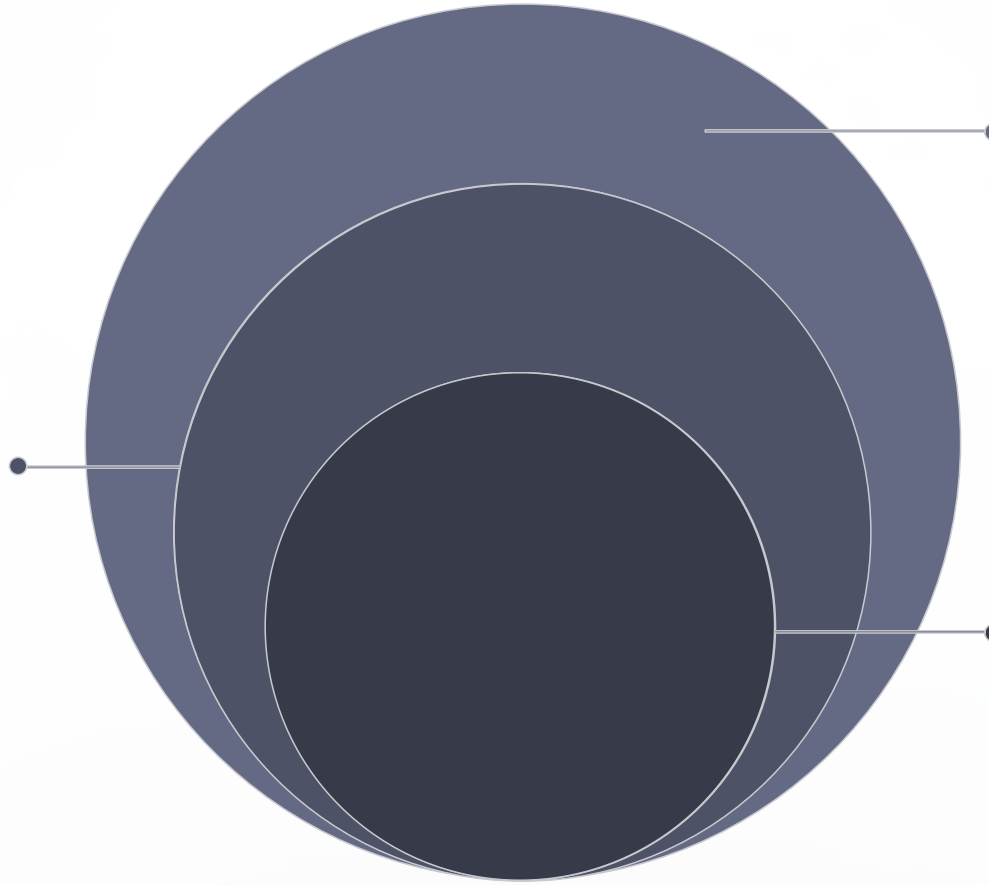
LLMs

- GPT-4.1
- Claude Sonnet
- Qwen / DeepSeek

Memory

- Redis
- Vector DB

Agent Infrastructure Diagram



Integration Layer

UI, API gateway, memory, tools, external services

Agent Controller

Orchestrates actions and planning

Core LLM

Generative model driving reasoning

The infrastructure connects every layer — from the user interface down to external services including **web search, databases, CRM, code execution,** and **browser automation** — enabling fully autonomous end-to-end workflows.

Agent Execution Loop

Observe

Perceive environment and inputs.

Reason

Analyze context and goals.

Plan

Determine the best sequence of actions.

Evaluate

Assess results and adjust.

Act

Execute tools and tasks.

Agents iterate through this loop continuously until the goal is fully achieved.

Agent Design Patterns Overview

01

ReAct Agents

Interleave reasoning and action steps.

02

Plan-and-Execute

Separate planning from execution phases.

03

Cognitive Loops

Continuous perception-to-memory cycles.

04

Tool-Using Agents

Structured calls to external APIs and tools.

05

Self-Reflection Agents

Agents that critique and improve their own outputs.

ReAct Architecture

ReAct = Reasoning + Action

The agent alternates between **Thought → Action → Observation** in a tight loop, grounding each reasoning step in real tool results before proceeding.

ReAct is the **foundation of modern agent frameworks** including LangGraph, AutoGPT, and CrewAI.

Example Loop

Thought

Search Google for NVIDIA stock price.

Action

Call finance API tool.

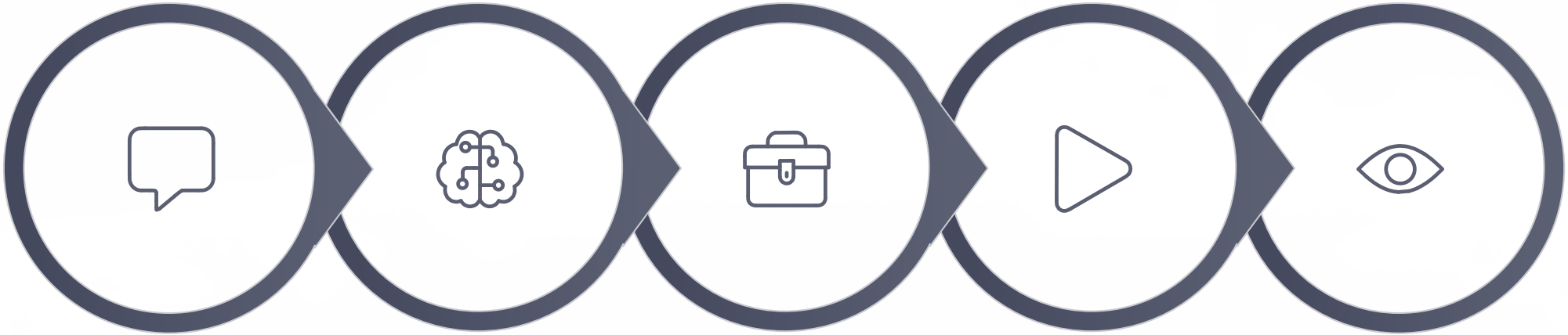
Observation

Receive price data and news results.

Next Thought

Summarize findings for the user.

ReAct Agent Diagram



Frameworks implementing ReAct: **LangGraph, AutoGPT, CrewAI**. The loop repeats until the agent produces a final, grounded answer backed by real observations.

ReAct in Action: A Real Example

> Question: "Find the latest NVIDIA stock price and summarize the news."

1

Thought

Search finance API for NVIDIA price.

2

Action

Call finance API tool.

3

Observation

Receive current stock price.

4

Action

Web search for NVIDIA news.

5

Final Answer

Summarized price + news delivered to user.

Plan-and-Execute Agents

Architecture

Planner Agent	Executor Agent
Creates a full task plan upfront.	Runs each step sequentially.

Advantages: **predictable, structured workflows** with clear separation of concerns.

Example: Write a Research Report

01

Search Sources

Planner identifies relevant papers and data.

02

Summarize Papers

Executor distills key findings.

03

Draft Report

Executor compiles the final document.

Plan-and-Execute Diagram

The clean separation between **Planner** and **Executor** makes these agents highly predictable and auditable — ideal for structured, multi-step enterprise workflows.

Cognitive Loop Architecture

Agents continuously improve decisions through a persistent perception-to-memory cycle, enabling **long-running intelligent systems**.

Perception

Sense environment inputs.

Reasoning

Analyze and interpret data.

Decision

Choose optimal action path.

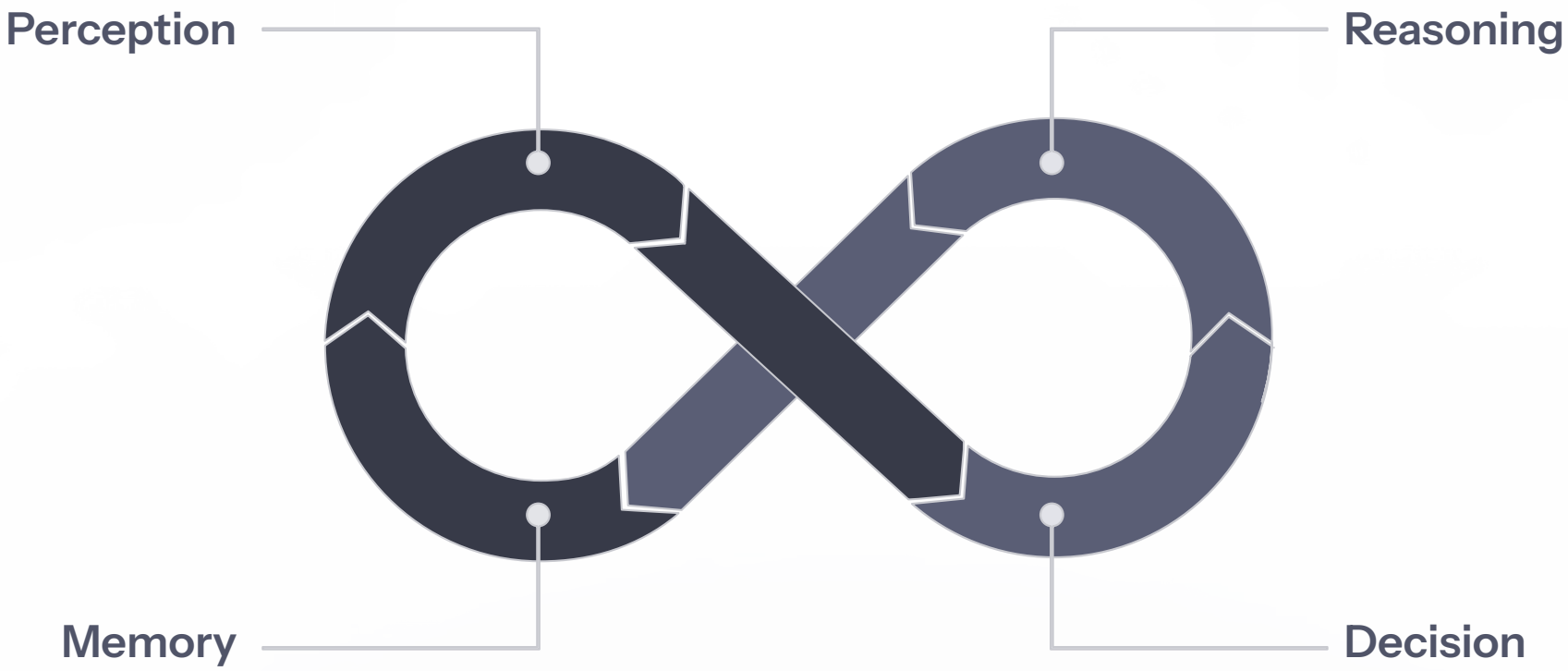
Action

Execute in the environment.

Feedback + Memory Update

Learn and persist context.

Cognitive Loop Diagram



The memory update step is what distinguishes cognitive loop agents from simple reactive agents — they **learn from every interaction** and carry that knowledge forward across the entire task lifecycle.

Tool-Using Agents

Agents interact with external systems via **structured tool calls**. Modern models natively support function calling with typed schemas.

Web Search

Real-time information retrieval.

Python Execution

Run code and return results.

APIs

Finance, weather, CRM, and more.

Database Queries

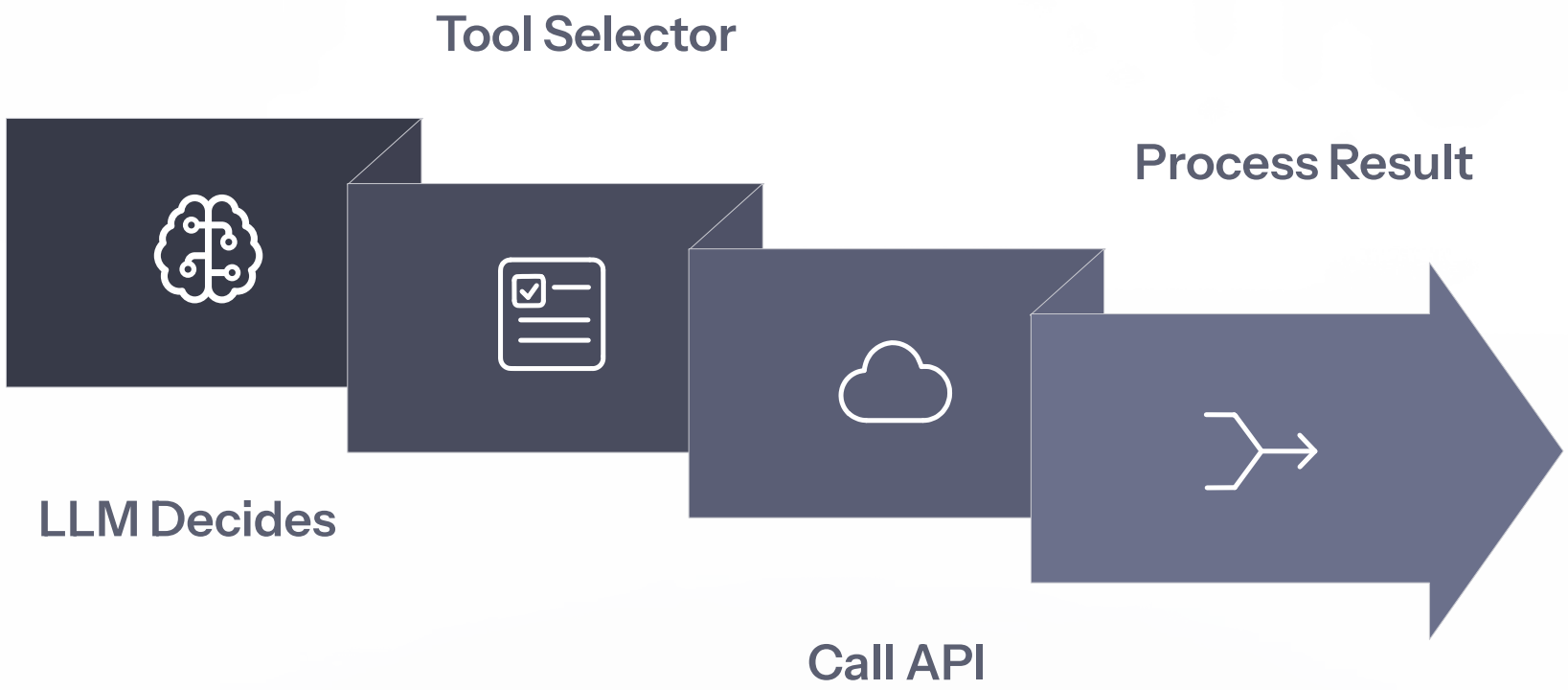
Read and write structured data.

Browser Automation

Navigate and interact with web pages.

Example: `{ "tool": "weather_api", "location": "Singapore" }`

Tool-Using Agent Diagram



The LLM acts as the **orchestrator** — it selects the right tool, interprets the result, and decides whether to call another tool or deliver a final answer to the user.

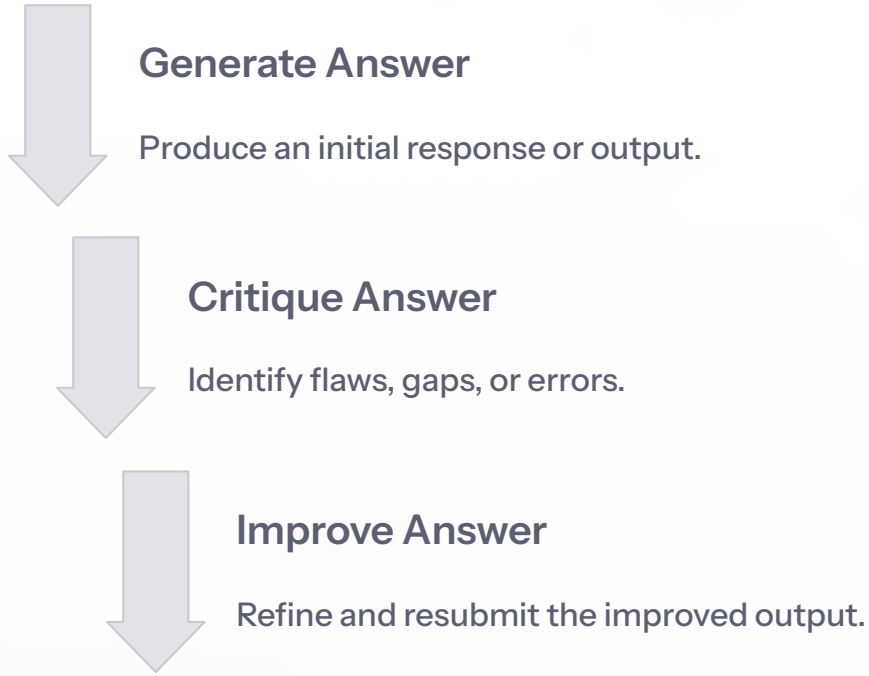
Self-Reflection Agents

The Core Idea

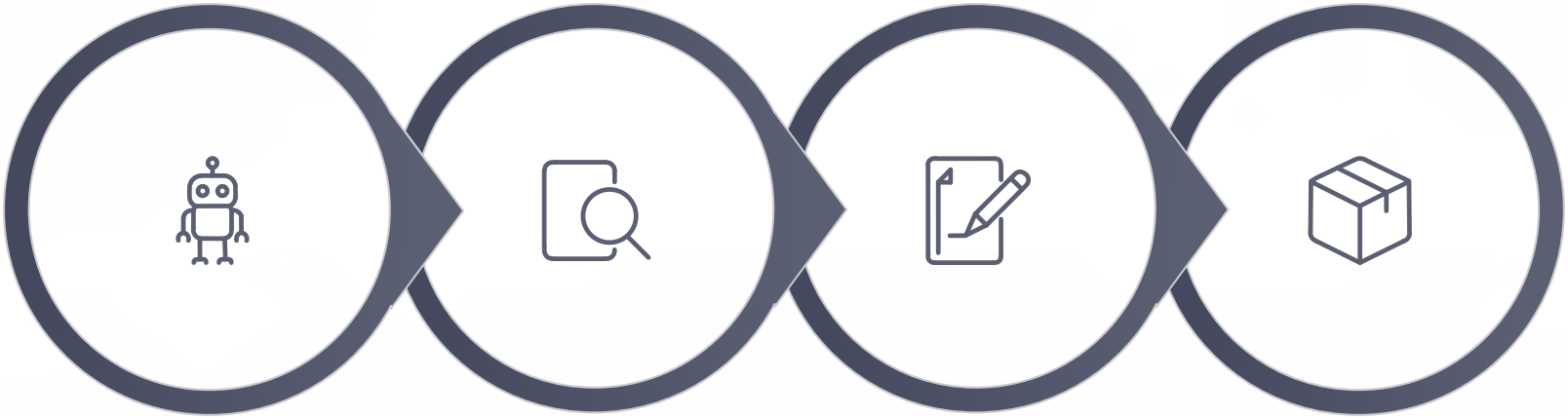
The agent **reviews and critiques its own outputs** before delivering them, iterating until quality meets a defined threshold.

Used in: **coding agents, research agents, planning systems**.

Reflection Loop



Reflection Architecture Diagram



A practical example: a **coding agent** generates code, a critic agent reviews it for bugs and style issues, and the improved version is submitted — all without human intervention.

Modern Prompting Strategies

Few-Shot Prompting

Provide examples to guide model behavior.

Chain-of-Thought

Encourage step-by-step reasoning.

Self-Consistency

Generate multiple answers, pick consensus.

System Prompts

Define model role and behavior upfront.

Structured Outputs

Force reliable, parseable JSON responses.

Few-Shot Prompting

How It Works

Provide labeled input-output examples directly in the prompt. The model learns the pattern from the examples and applies it to new inputs — no fine-tuning required.

Example: Translation

Prompt: Translate to French.

Hello → Bonjour
Good morning → Bonjour

The model learns the translation pattern and applies it to any new input.

Chain-of-Thought Prompting

Encourages the model to **reason step by step** before producing a final answer, dramatically improving accuracy on complex tasks.

Prompt Instruction

"Solve step by step. Explain your reasoning before giving the final answer."

Math Problems

Break arithmetic and algebra into explicit steps.

Logical Reasoning

Trace deductive chains clearly.

Planning Tasks

Decompose goals into ordered sub-steps.

Self-Consistency Reasoning

The model generates **multiple independent answers**, then selects the most common result — improving reasoning accuracy through consensus.

01

Generate 5 Answers

Run the same prompt multiple times with varied reasoning paths.

02

Compare Reasoning

Evaluate the logic and conclusions of each answer.

03

Pick Consensus

Select the most frequently occurring correct answer.

System Prompt Engineering

What System Prompts Do

System prompts **define model behavior, persona, and constraints** before any user interaction begins. They are the primary lever for shaping agent personality and output quality.

Used in: **enterprise AI agents, customer support, coding assistants**.

Example System Prompt

> You are a senior financial analyst. Use precise, structured answers. Always cite sources. Never speculate without data.

Well-crafted system prompts are the difference between a generic chatbot and a **reliable, role-specific AI agent**.

Structured Outputs (JSON Schema)

Agents produce **structured, machine-readable responses** using JSON schemas — enabling reliable parsing, automation pipelines, and seamless API integration.

Example Output Schema

```
{
  "summary": "...",
  "confidence": 0.95,
  "sources": []
}
```

Benefits

Reliable Parsing

Downstream systems consume outputs without ambiguity.

Automation Pipelines

Chain agent outputs directly into workflows.

API Integration

Plug agent results into any external service.

Modern LLM Ecosystem (2026)

OpenAI

GPT-4.1 · GPT-4.5

Anthropic

Claude 3 Opus · Claude Sonnet

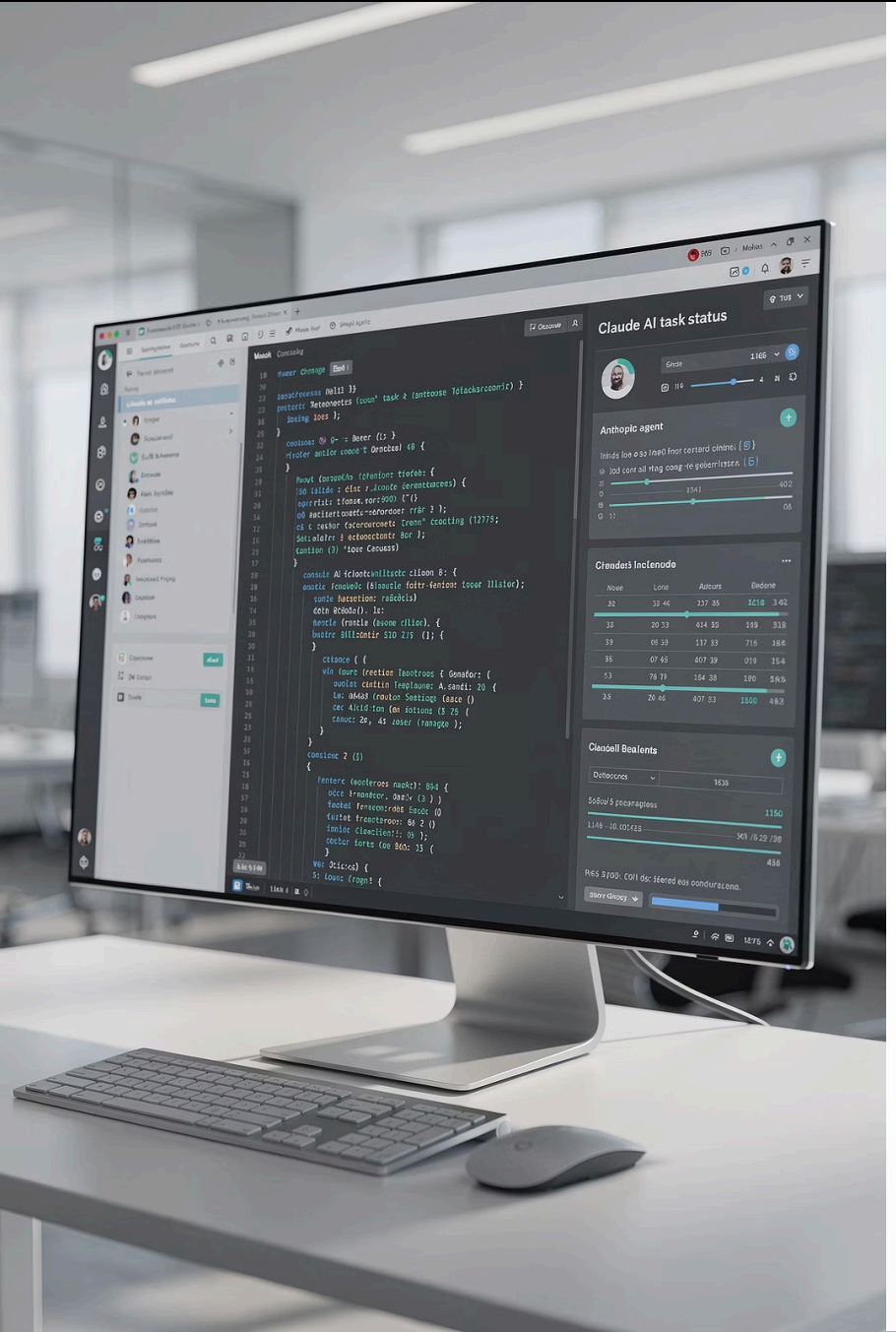
Alibaba

Qwen-2

DeepSeek

DeepSeek-R1 (reasoning-focused)

All leading 2026 models natively support **tool use and structured outputs**, making them production-ready for agentic deployments.



Claude Agent Tasks: Real Capabilities

Claude Opus models are purpose-built for **long-running agent tasks and complex reasoning workflows.**

1

Multi-Hour Coding Tasks

Sustain context and progress across extended sessions.

2

Tool Execution

Call APIs, run code, and interact with file systems.

3

Multi-Step Planning

Decompose and execute complex multi-stage goals.

4

File System Interaction

Read, write, and manage files autonomously.

Local LLM Agents with Ollama

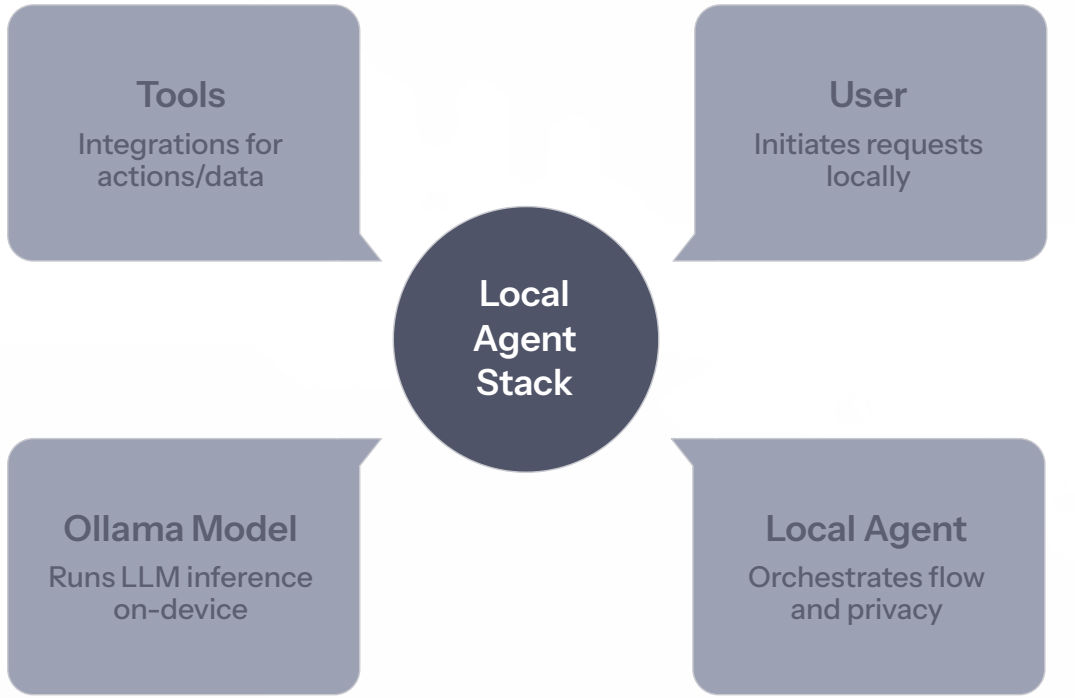
Run Models Locally

Deploy powerful LLMs on your own hardware using **Ollama** — no cloud dependency required.

Supported Models

- Qwen2
- Llama3
- DeepSeek

Local Agent Architecture



Privacy

Data never leaves your infrastructure.

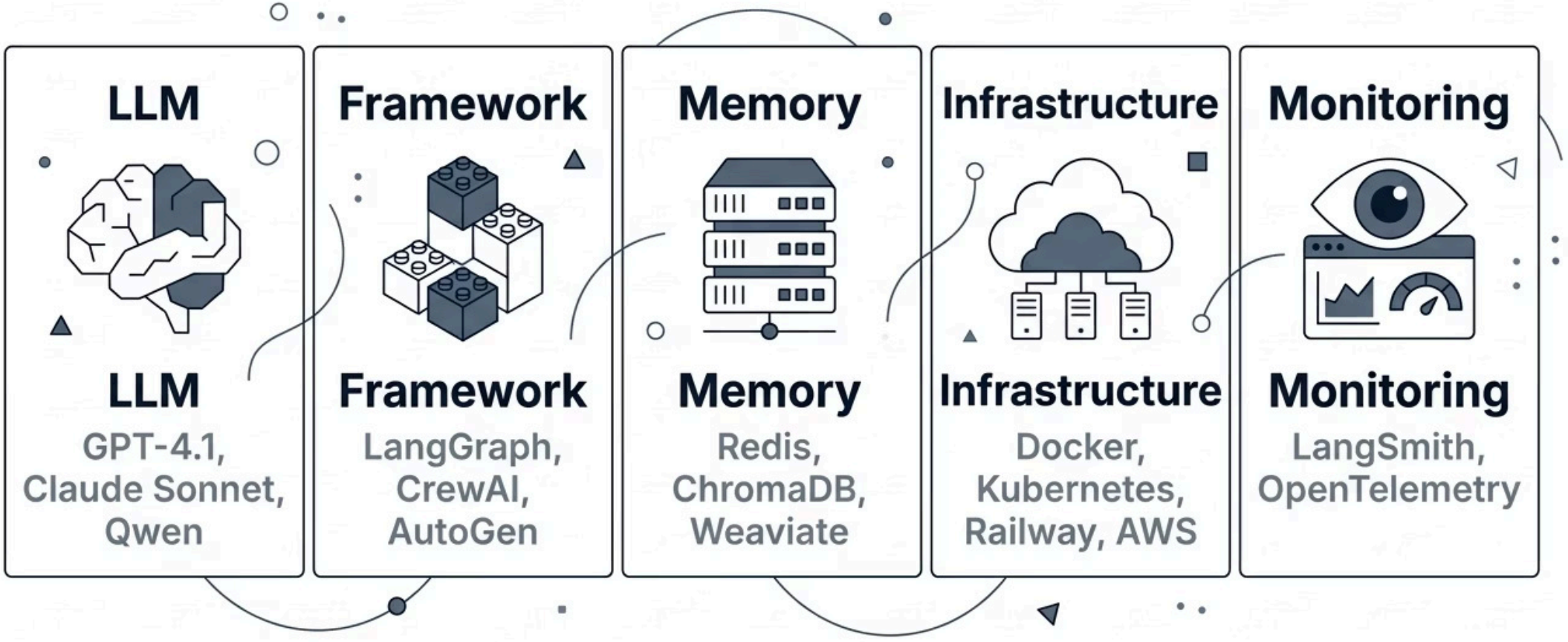
Low Cost

No per-token API fees.

Offline

Works without internet access.

Production AI Agent Stack (2026)



A robust production stack combines the right **LLM, orchestration framework, memory layer, cloud infrastructure, and observability tooling** to deliver reliable, scalable agentic systems.

Real-World AI Agent Use Cases

Automated Research Assistants

Search, synthesize, and report on any topic autonomously.

Coding Agents

Write, test, debug, and deploy code end-to-end.

Customer Service Agents

Handle queries, escalations, and resolutions 24/7.

Financial Analysis Agents

Monitor markets, generate reports, and flag anomalies.

DevOps Automation

CI/CD pipelines, incident response, and infrastructure management.

AI Copilots

Augment human workflows across every business function.